1. What is a stack?

A stack is a linear data structure where insertion and deletion happen at one end called the top. It follows LIFO: Last In, First Out.

Analogy: a pile of plates. The last plate placed on top is the first plate removed.

The most recently inserted element is the first one removed.

A stack is not intended for arbitrary indexing; normally you inspect only the top.

2. Stack operations in C++ STL

Operation	Purpose	Typical complexity
push(x)	Insert x at the top	O(1) amortized
pop()	Remove top element; returns nothing	O(1)
top()	Access top element	O(1)
empty()	Check whether empty	O(1)
size()	Number of elements	O(1)

Important:
Calling top() or pop() on an empty stack is invalid. Check empty() first whenever it may be empty.

3. Stack STL example

Include <stack>

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<int> st;
    st.push(10);
    st.push(20);
    st.push(30);

    cout << st.top() << '\n'; // 30
    st.pop();                 // removes 30
    cout << st.top() << '\n'; // 20
    cout << st.size() << '\n';// 2
    if (!st.empty()) st.pop();
}
```

**pop() does not return the removed element. If you need it, save top() before popping:

```
int removed = st.top();
st.pop();
```

4. Dry run

Command	Stack (bottom → top)	top()
push(4)	4	4
push(7)	4, 7	7
push(9)	4, 7, 9	9
pop()	4, 7	7
push(2)	4, 7, 2	2

5. Applications of stacks

Undo/redo: the most recent action is undone first.

Function calls and recursion: the call stack tracks active calls.

Balanced brackets: check correct nesting of parentheses, brackets, and braces.

Expression processing: postfix evaluation and expression conversion.

Monotonic-stack problems: next greater/smaller element patterns.

Reversing data: push items, then pop them in reverse order.

6. Example: reverse a string

```
#include <iostream>
#include <stack>
#include <string>
using namespace std;
```

```
int main() {
    string s = "hello";
    stack<char> st;
    for (char ch : s) st.push(ch);

    string reversed;
    while (!st.empty()) {
        reversed += st.top();
        st.pop();
    }
    cout << reversed; // olleh
}
```

Time: O(n), because each character is pushed and popped once. Auxiliary space: O(n), for the stack and result string.

7. Example: balanced round parentheses

```
bool isBalanced(const string& s) {
    stack<char> st;
    for (char ch : s) {
        if (ch == '(') st.push(ch);
        else if (ch == ')') {
            if (st.empty()) return false;
            st.pop();
        }
    }
    return st.empty();
}
```